

Самостійна робота №7: Тестування в Nest.js

Тема: Тестування в Nest.js

Мета: Навчитися писати unit-тести для сервісів та контролерів Nest.js, використовувати мокінг залежностей та supertest для тестування API-ендпоінтів.

Кількість годин: 4

Теоретичний матеріал

Підхід до тестування у Nest.js

Nest.js має вбудовану підтримку тестування на базі Jest. Архітектура Dependency Injection (DI) дозволяє легко замінювати реальні залежності на моки. Модуль `@nestjs/testing` надає утиліти для створення тестового оточення. Тести розміщуються поруч з файлами, які вони тестують: `users.service.spec.ts` для `users.service.ts`.

Unit-тестування сервісів

Для тестування сервісу створюється тестовий модуль через `Test.createTestingModule()`. Залежності сервісу (наприклад, репозиторій TypeORM) замінюються на моки. Типова структура:

```
const module = await Test.createTestingModule({
  providers: [
    UsersService,
    { provide: getRepositoryToken(User), useValue: mockRepository },
  ],
}).compile();

const service = module.get<UsersService>(UsersService);
```

Мок репозиторію створюється як об'єкт з методами `find`, `findOne`, `save`, `delete`, реалізованими через `jest.fn()`.

Unit-тестування контролерів

Контролери тестуються аналогічно - сервіси замінюються на моки. Тести перевіряють, що контролер правильно делегує виклики сервісу та повертає очікувані відповіді. Guards та Pipes можна замінити через `.overrideGuard()` та `.overridePipe()`.

Інтеграційне тестування з supertest

Для E2E-тестування API використовується бібліотека `supertest`. Створюється повний Nest-застосунок (або його частина), `supertest` відправляє HTTP-запити та перевіряє відповіді:

```
const app = moduleFixture.createNestApplication();
await app.init();

await request(app.getHttpServer())
  .get('/users')
  .expect(200)
  .expect((res) => {
    expect(res.body).toHaveLength(2);
  });
```

Мокінг залежностей

Основні патерни мокінгу: повний мок модуля (`jest.mock()`), часткові моки (`jest.spyOn()`), фабричні провайдери (`useFactory`), мок значення (`useValue`). Для TypeORM: `getRepositoryToken()` дозволяє замінити репозиторій. Для зовнішніх сервісів (HTTP, email) - створюються мок-провайдери.

Контрольні питання

1. Як створити тестовий модуль у Nest.js за допомогою `Test.createTestingModule()`?
2. Яким чином Dependency Injection спрощує тестування у Nest.js?
3. Як замокувати TypeORM-репозиторій у unit-тестах?
4. Чим відрізняється unit-тестування контролерів від тестування сервісів?

5. Як використовувати supertest для тестування API-ендпоінтів?
6. Як замінити Guard або Pipe на мок у тестовому модулі?
7. Які підходи до мокінгу зовнішніх HTTP-запитів існують у Nest.js?

Практичне завдання

Напишіть тести для CRUD-сервісу `TasksService` з методами `findAll`, `findOne`, `create`, `update`, `delete`:

1. Створіть мок репозиторію з відповідними методами.
2. Напишіть unit-тести для кожного методу сервісу (мінімум 5 тестів).
3. Напишіть unit-тести для `TasksController`, перевіряючи делегування до сервісу.
4. Напишіть інтеграційний тест з supertest для ендпоінтів `GET /tasks` та `POST /tasks`.

Порядок виконання

1. Вивчити теоретичний матеріал, наведений у цьому документі.
2. Ознайомитись з рекомендованими джерелами.
3. Відповісти на контрольні питання.
4. Виконати практичне завдання.

Рекомендовані джерела

- [Nest.js - Testing](#) - офіційна документація по тестуванню
- [Jest - документація](#) - офіційна документація Jest
- [supertest - GitHub](#) - бібліотека для тестування HTTP
- [Nest.js - Unit Testing \(приклад\)](#) - рецепти тестування
- [NestJS Testing - Best Practices](#) - утиліти та приклади тестування